Target OS: Windows 10/11 **Database: Cloud Supabase PostgreSQL** **Repo: R41YAN/smart-id-system**

This guide provides complete instructions to set up, configure, and execute the Smart ID Card Formulation System (SIFS) on another machine (e.g., your friend's PC) to present the working project and automated testing suite to your teacher or evaluation panel.

ARCHITECTURE ADVANTAGE
SIFS uses a cloud-hosted PostgreSQL database on Supabase. Your friend DOES NOT need to install PostgreSQL, configure pgAdmin, or run local database seeds. The backend connects directly to the Supabase cloud cluster over the internet.

STEP 1: PREREQUISITES INSTALLATION

Ensure your friend's computer has the following two tools installed before proceeding:

- **1. Node.js (LTS Version):** Download Node.js 18.x or 20.x LTS from https://nodejs.org. Run the installer with default settings. Node.js includes npm.
- **2. Git for Windows:** Download from https://git-scm.com. Standard installer settings are completely fine.

FAQ: DOES YOUR FRIEND NEED TO CONFIGURE GIT AFTER DOWNLOADING?
NO! Default installation settings are 100% sufficient. Setting up git config (user.name and user.email) is ONLY required if your friend wants to commit or push code. For cloning, building, running, and testing, ZERO git configuration is required.

STEP 2: CLONE THE PROJECT REPOSITORY

Open Command Prompt (cmd) or PowerShell on your friend's computer, navigate to the desired working folder (e.g., Desktop or Projects), and clone the repository:

```
git clone https://github.com/R41YAN/smart-id-system.git
cd smart-id-system
```

STEP 3: BACKEND ENVIRONMENT SETUP (.ENV)

A. Create backend/.env: Navigate into the backend directory and create a configuration file named .env:

```
cd backend
# On Windows PowerShell, create the file using notepad:
notepad .env
```

B. Environment Variables: Paste the following environment variables into the backend/.env file and save it:

```
PORT=5000
DATABASE_URL=postgresql://postgres.qwqnrvpubmnarseleegr:[YOUR_PASSWORD]@aws-0-ap-
southeast-1.pooler.supabase.com:5432/postgres
JWT_SECRET=smart_id_super_secret_key_2026
NODE_ENV=development
```

CRITICAL SECURITY: REPLACE [YOUR_PASSWORD]
The repository code and this guide replace the real database password with [YOUR_PASSWORD] to protect your Supabase cluster from unauthorized access on public Git. You (the repository owner) must supply the real password privately to your friend when filling the .env file.

EXPLANATION: WHAT IS JWT_SECRET?
JWT_SECRET is a cryptographic signing key used by JSON Web Token (JWT) during administrator authentication. When an admin logs in, the backend signs the session token with this secret key. The default value (smart_id_super_secret_key_2026) is fully functional for local demonstration.

STEP 4: INSTALL BACKEND DEPENDENCIES & LAUNCH SERVER

Inside the backend/ directory, install the required packages and start the Node.js API server:

```
# Inside smart-id-system/backend
npm install
npm run dev
```

Expected Console Output: You should see confirmation messages indicating successful initialization:

```
[nodemon] starting `node server.js`
Server running on port 5000
Database connected successfully to Supabase PostgreSQL
```

STEP 5: INSTALL FRONTEND DEPENDENCIES & START CLIENT

Leave the backend terminal running. Open a SECOND terminal window or tab, navigate to the frontend folder, and launch the Vite development server:

```
# Open a new PowerShell / Command Prompt window
cd smart-id-system/frontend
npm install
npm run dev
```

Expected Console Output: Vite will output the local development URL. Open your web browser and navigate to:

```
  VITE v5.x.x  ready in 320 ms
  'œ  Local:   http://localhost:5173/
```

STEP 6: TEACHER PRESENTATION & DEFENSE DEMONSTRATION

Follow this exact workflow to demonstrate the full capabilities of the system to your course instructor:

1. Administrative Authentication: Navigate to http://localhost:5173/login. Enter your administrator email and password to access the administrative dashboard.

2. Student Admission & ID Formulation: Demonstrate the student registration form. Input Department (CSE), Academic Year (2026), Batch (05), and Roll (001). Show how the unique Smart ID (2026CSE05001) is formulated deterministically following university business logic.

3. Digital ID Card Generation: Show the generated digital ID card containing the student profile picture, formatted barcode/QR code, academic credentials, and official stamp. Demonstrate one-click PDF/card export.

4. Real-Time Cloud Persistence: Show that any student admitted on your friend's PC is instantly stored in the Supabase cloud PostgreSQL database and remains persistent across sessions.

STEP 7: AUTOMATED TESTING DEMONSTRATION (CO3 LAB REQUIREMENT)

Your course teacher will likely want to verify the software quality, unit testing, and test coverage implemented for the project. Run the automated test suites in front of the teacher:

A. Backend Testing (34 Unit & Integration Tests): Open terminal in backend/ and run the Jest test suite:

```
cd smart-id-system/backend
npm test
```

To display the comprehensive code coverage report (100% coverage on core ID generator engine):

```
npm run test:coverage
```

B. Frontend Component Testing (11 Component Tests): Open terminal in frontend/ and execute the Vitest test suite:

```
cd smart-id-system/frontend
npm test
```

To display the frontend coverage table:

```
npm run test:coverage
```

STEP 8: TROUBLESHOOTING & COMMON QUESTIONS

• **Database Connection Refused / Timeout:** Ensure your friend's PC has an active internet connection. Verify that [YOUR_PASSWORD] was replaced with the actual database password and that there are no accidental brackets or trailing spaces in backend/.env.

• **PowerShell Script Execution Policy Error:** Open PowerShell as Administrator and execute: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass. This allows npm and nodemon scripts to run without restriction.

• **Port Already in Use (EADDRINUSE):** Another service is occupying port 5000 or 5173. Run taskkill /F /IM node.exe in Command Prompt to terminate background node instances, then restart.

• **Can my friend edit or push to my GitHub repo?** Only if you invite them as a GitHub Collaborator or if they fork the repository. By default, cloning allows them to pull, build, run, and test locally without any ability to modify your remote GitHub repository.

• **Does my friend need a Supabase account?** No! Your friend uses the cloud database connection via backend/.env without ever logging in to Supabase. Only you (the project owner) need the Supabase admin login.

READY FOR DEFENSE

Your project is fully prepared for demonstration. All database persistence, authentication, and automated test coverage are self-contained and ready.